



Secure File Transfer Monitoring System

Design, Implementation, and Evaluation of a Policy-Based Data Loss Prevention Monitoring
Application

Sunny Rajesh Hudda

Cyber Security Internship

Project Report

Institution: Unified Mentor

Date: May 2026 – July 2026

Abstract

This report documents the design and implementation of a Secure File Transfer Monitoring System, a Django-based web application created to detect risky movement of sensitive files inside a local monitored workspace. The project simulates a data loss prevention and security operations workflow by allowing an administrator to define monitoring policies, classify sensitive files, generate risk-based alerts, assign investigations, record analyst notes, and escalate unassigned alerts by email. The system uses Python, Django, SQLite, file-system monitoring, hash calculation, role-based access control, and optional SMTP configuration to demonstrate how security evidence can be collected and managed in a structured way. The report follows APA 7 student-paper formatting and includes project objectives, requirements, architecture, database design, workflows, usage instructions, testing evidence, limitations, and future development recommendations. The final implementation is intended for a cybersecurity portfolio, lab demonstration, and educational presentation rather than production deployment.

Keywords: secure file transfer monitoring, data loss prevention, Django, SQLite, alert escalation, role-based access control

Table of Contents

Abstract

1. Introduction

2. Problem Statement

3. Project Objectives

4. Scope and Assumptions

5. Background and Standards Alignment

6. Functional and Nonfunctional Requirements

7. System Design

8. Architecture and Diagrams

9. Implementation Details

10. How to Use the System

11. Testing and Validation

12. Security Considerations

13. Database Requirement

14. Screenshot Placeholders

15. Future Developments

16. Conclusion

References

1. Introduction

Organizations frequently move confidential files between folders, devices, cloud-sync directories, network shares, and temporary workspaces. When those transfers are not monitored, sensitive data such as payroll exports, identity documents, client data, source-code archives, or database backups can leave approved locations without timely review. The Secure File Transfer Monitoring System was created to demonstrate how a security operations team could detect and manage these events using a policy-driven application.

The project was implemented as a Django web application with a local SQLite database, a file-monitoring worker, an alert escalation worker, and optional SMTP email integration. Django was selected because it provides a structured web framework with built-in authentication capabilities, while SQLite was selected because it is lightweight, file-based, and appropriate for a local lab deployment (Django Software Foundation, n.d.-a; SQLite Consortium, n.d.). The system is not intended to replace enterprise-grade DLP products. Instead, it demonstrates core ideas that appear in real security tooling: policy definition, classification logic, evidence collection, role-based investigation, email notification, and audit-ready reporting.

2. Problem Statement

The problem addressed by the project is the lack of visibility over sensitive file movement in a small organization or lab environment. A user may copy a salary file to a USB-like folder, move a client CSV to a cloud-sync directory, or transfer a source-code archive to an unapproved location. Without monitoring, the event may not be noticed until after data exposure has occurred. A practical monitoring system should identify sensitive files, determine whether the destination is approved, calculate useful evidence such as file hashes, generate an alert, assign ownership, and record the response actions taken by analysts.

The system is designed around the assumption that early detection and structured response reduce investigation delay. NIST incident response guidance emphasizes preparation, detection and analysis, containment, eradication, recovery, and post-incident activity as important stages of handling security events (Cichonski et al., 2012). This project maps those ideas into a small web-based workflow: the policy prepares the environment, the monitor detects file activity, the dashboard supports analysis, the analyst resolves or escalates the alert, and the report preserves the evidence.

3. Project Objectives

The primary objective of the project is to create a working security monitoring web application that can be demonstrated in a lab video and explained as a portfolio project. The application should provide clear visual dashboards, simple alert details, and a realistic response process. Secondary objectives include supporting user roles, scoped policy access, password reset through OTP email, automatic escalation emails for unassigned alerts, and basic reporting. The system should also avoid shipping old test data or credentials by excluding the SQLite database and .env file from the final project package.

Table 1

Project objectives and implementation evidence

Objective	Implementation in the project	Evidence to show in screenshots
Detect risky sensitive file movement	File monitor checks monitored directories, sensitive folders, restricted filenames, extensions, blocked destinations, and allowed destinations.	Policy page, runmonitor terminal, generated alert.
Create readable alerts	Alert detail page explains what happened, why it is risky, ownership, file path, and recommended response.	Alert detail screenshot.
Support investigation workflow	Analyst can claim alert, update status, add notes, and close the case.	Claimed alert and resolved status.
Escalate neglected alerts	Escalation worker checks unassigned alerts and sends email when the SLA time is exceeded.	Email Alerts page and worker terminal.
Support password reset	OTP reset flow uses registered email and expiry settings from .env.	Forgot password and OTP pages.
Preserve audit evidence	Events, alerts, email notifications, integrity history, and reports are stored in SQLite.	Dashboard, reports, and database discussion.

4. Scope and Assumptions

The project scope is limited to local folder monitoring and educational security operations simulation. The application runs on a developer workstation and monitors folders inside a configured workspace. The default workspace contains folders such as Sensitive, Approved, USB_Drive, CloudSync, and NetworkShare. These folders simulate realistic movement destinations without requiring physical USB devices or cloud-provider integration.

The system assumes that the user running the monitor has permission to read the folders being watched. Process attribution is best-effort because operating systems may restrict access to other process handles. The system also assumes that email delivery depends on correct SMTP credentials in the .env file. If credentials are not configured, email behavior can be tested locally, but real inbox delivery will not occur.

5. Background and Standards Alignment

The design uses several security concepts that align with common practices in application security and incident response. OWASP ASVS provides a basis for verifying application security controls such as authentication, access control, logging, and data protection (OWASP Foundation, n.d.-a). The project implements these concepts at a lab level by separating superuser, policy admin, analyst, and auditor roles; recording security evidence; and requiring login before access to operational pages.

The system also incorporates incident-handling concepts from NIST SP 800-61 Revision 2. In a real environment, security events require detection, analysis, prioritization, documentation, response, and follow-up review (Cichonski et al., 2012). The project demonstrates these steps through alert triage, ownership, notes, closure, and reporting. Hashing

also supports evidence consistency because Python hashlib provides secure hash and message-digest algorithms such as SHA-256 (Python Software Foundation, n.d.-b).

Django was used for the web application layer because it provides a mature framework for routing, forms, templates, sessions, and authentication. Django documentation describes its authentication system and email framework, both of which are used in this project for login, role access, OTP reset, and SMTP-based notifications (Django Software Foundation, n.d.-a, n.d.-b). Python pathlib supports clean path handling across operating systems, which is useful when monitoring directories in a portable lab project (Python Software Foundation, n.d.-a).

6. Functional and Nonfunctional Requirements

Functional requirements describe what the system must do. Nonfunctional requirements describe how the system should behave. Because this project is designed for demonstration and learning, clarity, stability, and ease of setup are more important than enterprise scale.

Table 2

Functional requirements

ID	Requirement	Description
FR-1	Authentication	Users must log in before accessing dashboards and alert data.
FR-2	Role-based views	Superuser, admin, analyst, and auditor accounts should see appropriate functions.
FR-3	Policy management	Admins should define monitored paths, sensitive rules, allowed destinations, and escalation settings.
FR-4	File monitoring	The worker should detect file creation, movement, modification, deletion, and manual scans.
FR-5	Risk classification	Events should be classified using folder paths, filenames, extensions, and destination keywords.
FR-6	Alert ownership	Analysts should claim unassigned alerts and update investigations.
FR-7	Email escalation	Unassigned alerts should be emailed to the configured escalation group after the policy SLA.
FR-8	Password reset	Users should request password reset through OTP sent to registered email.
FR-9	Reporting	Events and alerts should be visible on dashboards and exportable for evidence.

7. System Design

The system follows a modular design. The web layer provides the user interface and role-based access. The monitoring layer observes folders and sends events to the classification

engine. The classification engine calculates risk and creates evidence. The database layer stores users, policies, alerts, and investigation notes. The email layer sends OTP and alert notifications when SMTP settings are present. The separation of these components makes the application easier to explain and maintain.

A key design choice is to keep the final zip clean. The package should contain source code, templates, static files, requirements, workspace folders, migrations, and `.env.example`. It should not include `db.sqlite3`, real `.env` credentials, or pre-created users and policies. This prevents old test data and private credentials from being shared. Users generate their own database by running migrations and then create their own superuser, users, and policies.

8. Architecture and Diagrams

The following diagrams document the major design views of the application. They can be used in the final presentation and can also guide future development.

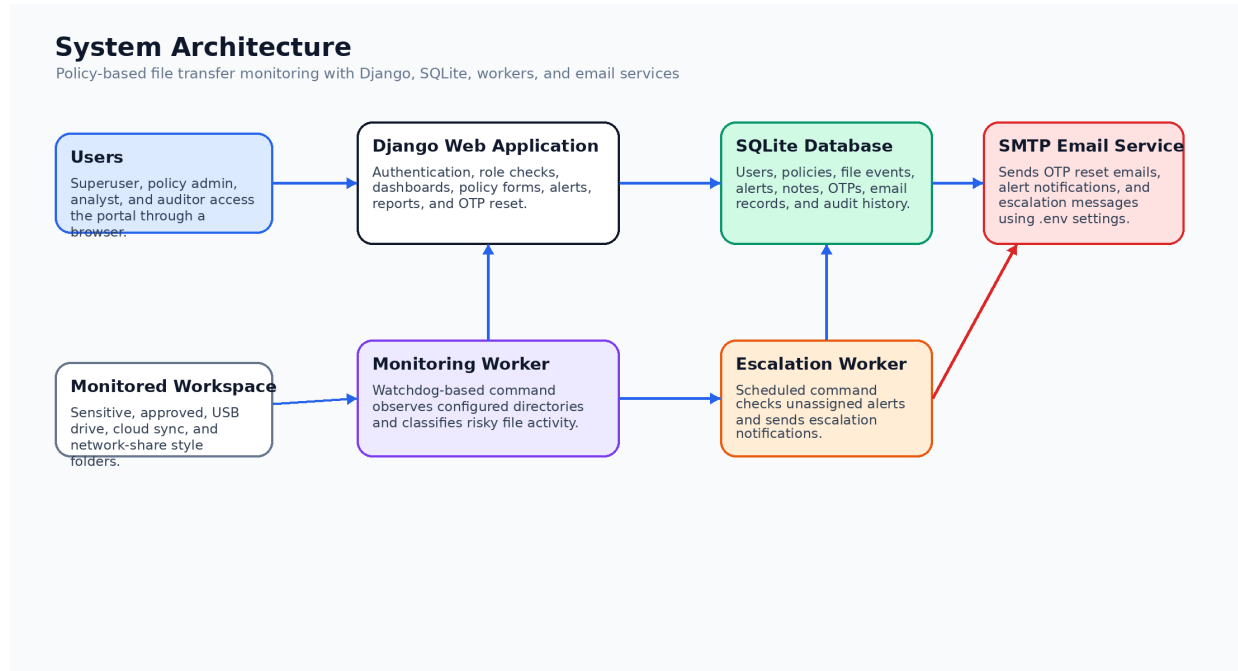


Figure 1: System architecture diagram.

Note. The web application, database, monitoring worker, escalation worker, email service, and monitored workspace are separated to show component responsibility.

File Monitoring and Alert Workflow

From file movement to alert triage and resolution



Outcome: closed alerts are removed from active dashboard counts, while the evidence remains available for audit and reports.

Figure 2: File monitoring and alert workflow.

Note. The workflow shows how a configured policy leads to file event detection, classification, evidence creation, analyst investigation, and closure.

Role-Based Access Model

Separation of duties between system administration, investigation, and evidence review

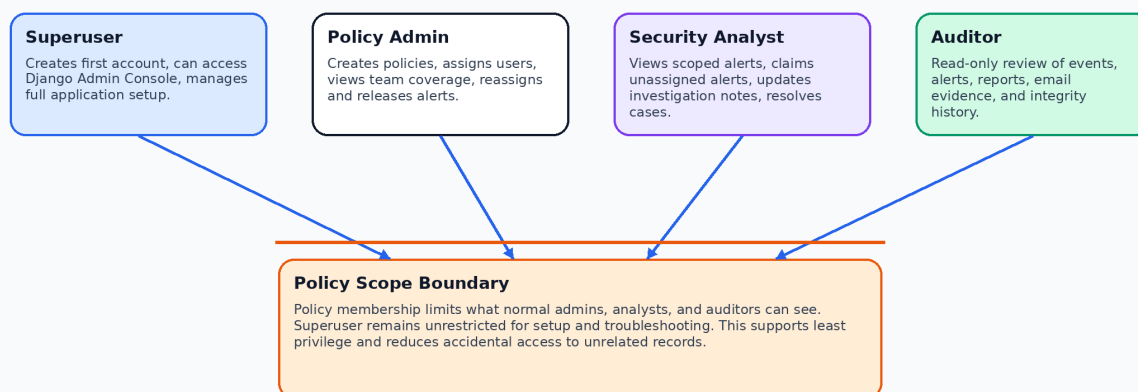


Figure 3: Role-based access control model.

Note. The diagram shows how superuser access differs from policy admin, analyst, and auditor access.

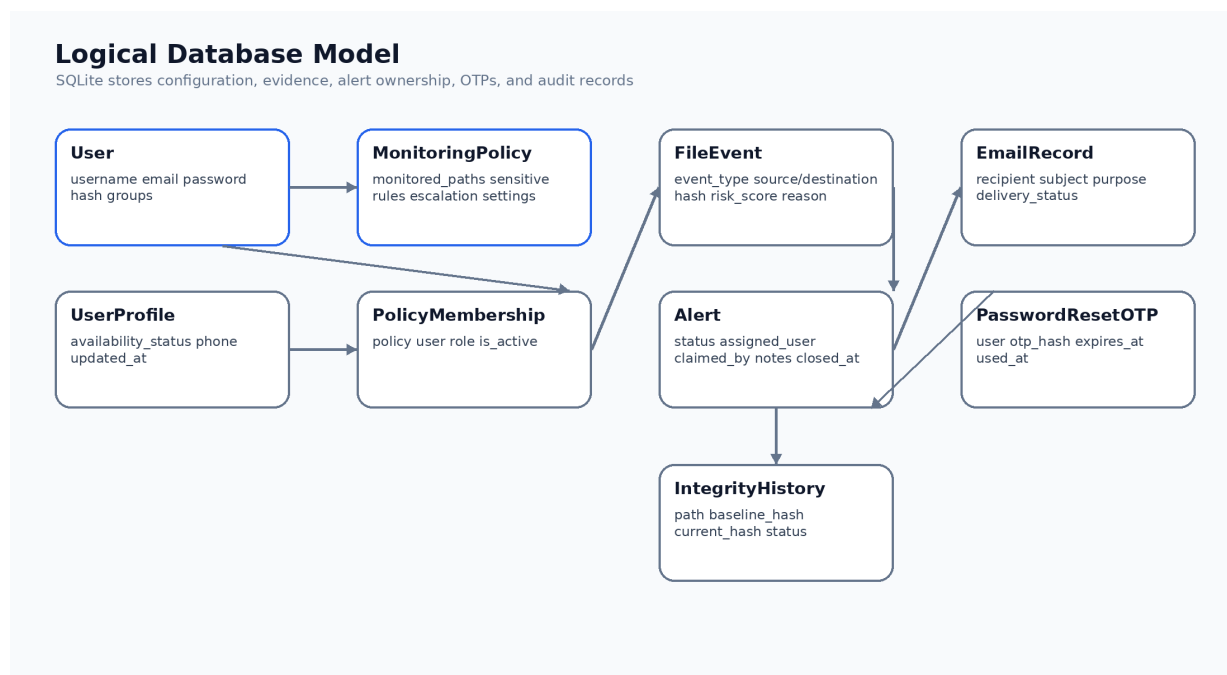


Figure 4: Logical database model.

Note. The diagram is logical rather than a full physical schema. It shows the major records needed for users, policies, evidence, alerts, email, OTP reset, and integrity history.

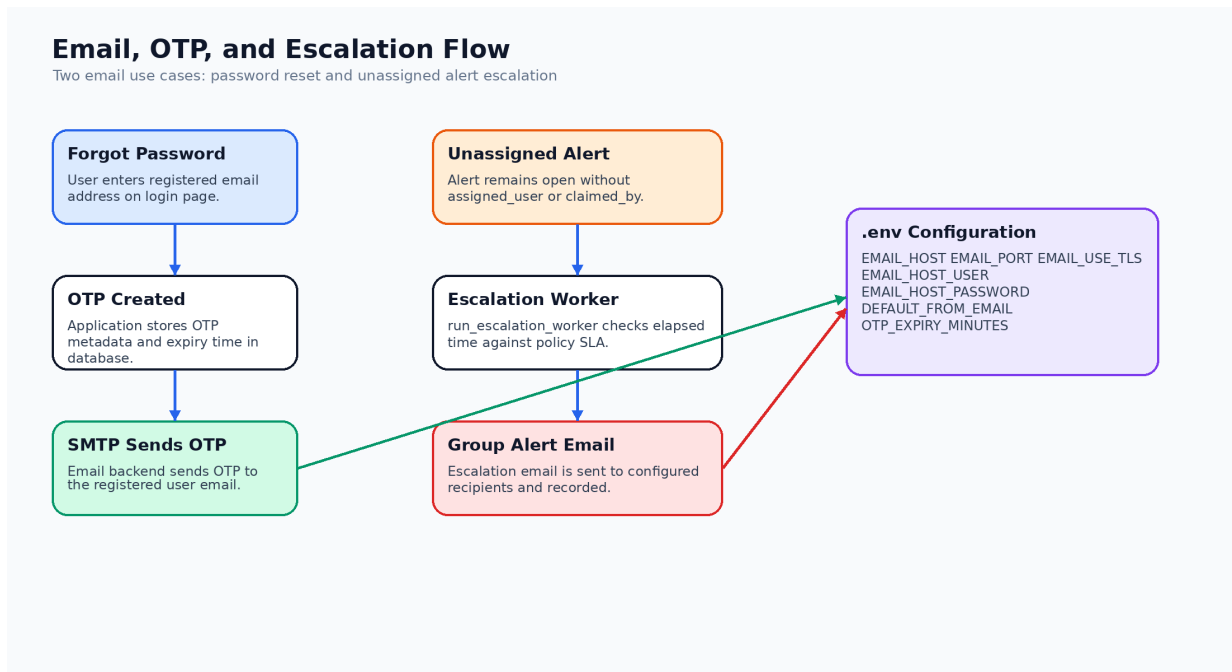


Figure 5: Email, OTP, and alert escalation flow.

Note. The diagram separates the password reset path from the alert escalation path and shows the role of .env SMTP configuration.

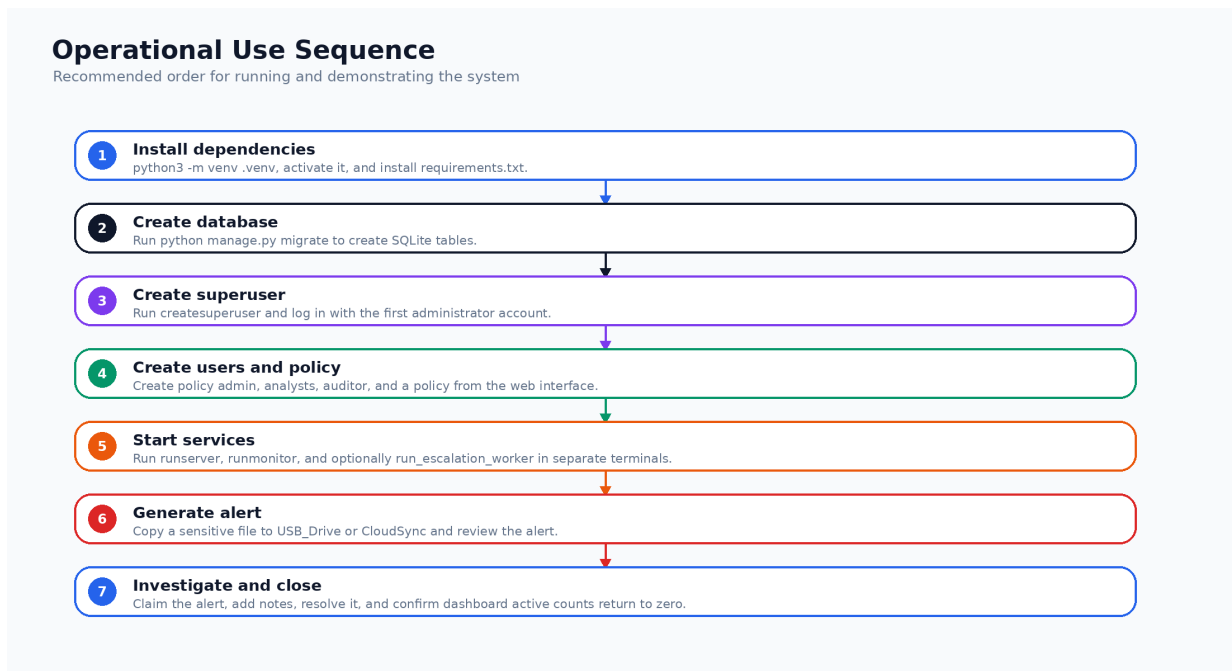


Figure 6: Operational use sequence.

Note. The diagram shows the recommended order for installing, creating the database, creating users, starting services, generating alerts, and closing investigations.

9. Implementation Details

9.1 Technology Stack

The backend is implemented in Python with Django. The database is SQLite. The application uses Django models to represent users, policies, events, alerts, email records, OTP records, and integrity results. The dashboard and pages are rendered using Django templates and static CSS. Management commands run background tasks such as file monitoring, integrity scanning, and escalation checks.

Table 3

Technology stack

Layer	Technology	Purpose
Web framework	Django	Routes, views, templates, forms, authentication, and role-protected pages.
Programming language	Python	Application logic, file handling, hashing, command-line workers, and reporting.
Database	SQLite	Local storage for configuration, events, alerts, users, OTPs, and audit records.
File paths	pathlib	Cross-platform path handling for monitored directories and workspaces.
Hashing	hashlib	SHA-256 or MD5 hash calculation for file evidence and integrity checks.
Email	Django email backend	SMTP delivery for OTP reset, alert notifications, and escalation emails.

9.2 Policy Engine

The policy engine defines what should be monitored and what should be considered risky. A policy can include monitored directories, sensitive directories, allowed destinations, restricted files, sensitive filename keywords, sensitive extensions, blocked destination keywords, trusted process keywords, hash algorithm selection, burst-transfer thresholds, notification

recipients, and escalation settings. This allows one policy to represent a department, application, or business process.

When a file event is detected, the classification logic checks whether the file is inside a sensitive area, whether the filename matches restricted patterns, whether the extension is commonly sensitive, and whether the destination contains blocked keywords such as `USB_Drive` or `CloudSync`. If a sensitive file appears outside an approved destination, the event becomes unauthorized and receives a higher risk score. If the risk crosses the alert threshold, the system creates an alert.

9.3 Alert Detail Page

The alert detail page was simplified to make the information easier to understand. Instead of showing only raw technical fields, the page is organized around practical investigation questions: What happened? Why is it risky? Who owns the alert? Has the alert been escalated? What file path was involved? What should the analyst do next? This format is more realistic for junior analysts and easier to demonstrate in a project video.

9.4 Password Reset and Email Notifications

The password reset feature uses an OTP workflow. A user requests a reset by entering the email address registered on the account. The system creates an OTP with an expiry time and sends it through the configured SMTP backend. Real email delivery requires a `.env` file with `EMAIL_HOST`, `EMAIL_PORT`, `EMAIL_USE_TLS`, `EMAIL_HOST_USER`, `EMAIL_HOST_PASSWORD`, and `DEFAULT_FROM_EMAIL`. Django supports multiple email backends, including SMTP for delivery and console output for local development (Django Software Foundation, n.d.-b).

Alert escalation uses a separate worker. When an alert remains open and unassigned beyond the policy escalation time, the worker sends an escalation email to configured recipients. The system also records notification evidence in the database. This creates an audit trail showing whether the alert was sent, who received it, and whether delivery succeeded or failed.

9.5 Dashboard and Reporting

The dashboard presents active alerts, unassigned alerts, severity mix, ownership coverage, seven-day activity trends, team availability, and recent file activity. Closed and false-positive alerts are removed from active counts, but the evidence remains stored for audit and reporting. This distinction is important because active workload should not include resolved cases, while reports still need historical evidence.

10. How to Use the System

10.1 Initial Setup

The project should be extracted into a folder and run from the command line. The following setup creates a virtual environment, installs dependencies, creates a .env file from the example, creates the database, and starts the server. The .env file should be edited only on the local machine and should never be submitted with the project because it contains secrets.

Table 4

Basic setup commands

Step	Command	Purpose
1	<code>python3 -m venv .venv</code>	Create an isolated Python environment.
2	<code>source .venv/bin/activate</code>	Activate the virtual environment on macOS/Linux.
3	<code>pip install -r requirements.txt</code>	Install Django and required packages.
4	<code>cp .env.example .env</code>	Create local environment configuration.
5	<code>python manage.py migrate</code>	Create SQLite database tables.
6	<code>python manage.py createsuperuser</code>	Create the first administrative account.
7	<code>python manage.py runserver</code>	Start the web application.

10.2 Email Setup

To enable real password reset and alert escalation email, the `.env` file must contain SMTP settings. The sender account should use an app password when using Gmail. The application reads these values when Django starts, so the server must be restarted after `.env` changes. The user email entered on the forgot-password page must match the email stored on the user account.

10.3 Creating Users and Policy

After logging in as superuser, the user should create a normal policy admin, analysts, and auditors from the Users/Roles page. Then the policy admin or superuser should create a monitoring policy. The policy should include a monitored directory, a sensitive directory, an approved destination, restricted filenames, sensitive keywords, sensitive extensions, blocked destination keywords, and escalation recipients. This manual setup keeps the project clean and helps demonstrate understanding of how policies and users are connected.

10.4 Starting Monitoring and Escalation

The web server, file monitor, and escalation worker should be run in separate terminals. The web server displays the application. The file monitor observes file activity. The escalation worker checks whether open alerts are still unassigned after the policy SLA.

Table 5

Operational commands

Service	Command	Description
Web application	<code>python manage.py runserver</code>	Starts the Django site at <code>http://127.0.0.1:8000/</code> .
File monitoring	<code>python manage.py runmonitor</code>	Watches active policy paths and creates events/alerts.
Escalation worker	<code>python manage.py run_escalation_worker --interval 30</code>	Checks unassigned alerts every 30 seconds.
Integrity scan	<code>python manage.py scan_integrity</code>	Creates or checks hash baselines for monitored files.

10.5 Testing a File Transfer Alert

To test the system, copy a file from the Sensitive folder into the USB_Drive or CloudSync folder after the monitor is running. The application should record the file event, classify it as risky, create an alert, and display it on the dashboard. The analyst should claim the alert, review the details, write notes, and close it. Once closed, the active alert count should return to zero while the historical evidence remains available.

11. Testing and Validation

Testing focused on confirming that the core system works after a fresh installation. The most important validation requirement is that the final project zip does not depend on pre-existing users, policies, or database data. The database should be created through migrations, and the user should configure policies manually from the interface. Functional tests should be repeated before final submission and during the demonstration video.

Table 6

Validation test plan

Test case	Expected result	Evidence to capture
Fresh migration	Database tables are created without errors.	Terminal screenshot showing migration success.
Superuser login	Superuser reaches dashboard and sees Django Admin link.	Login and sidebar screenshot.
Normal admin login	Normal admin sees policy/user pages but not Django Admin link.	Sidebar screenshot.
Create policy	Policy saves monitored and sensitive folders correctly.	Policy page screenshot.
Start monitor	runmonitor begins watching active policy paths.	Terminal screenshot.
Risky copy to USB_Drive	Alert is created with clear reason and risk score.	Dashboard and alert detail screenshots.
Claim alert	Analyst becomes owner and status changes to investigating.	Alert ownership screenshot.
Close alert	Active alert count returns to zero.	Before/after dashboard screenshots.
Forgot password OTP	OTP email is sent to registered email when SMTP is configured.	Inbox or Email Alerts page screenshot.
Unassigned escalation	Alert email is sent if no owner claims the alert within SLA.	Email record and terminal screenshot.

12. Security Considerations

The application demonstrates important security controls but remains a lab project. The final package should not contain real credentials, real email app passwords, or `db.sqlite3`. The `.env` file should be local only and excluded from source control. Superuser access should be limited to initial setup and troubleshooting. Normal admins should manage policies and users through the application interface rather than through the Django Admin Console.

Role separation supports least privilege. Analysts should only update alerts they own or are assigned to. Auditors should have read-only access to logs, alerts, and reports. Password reset should depend on registered email addresses and expiring OTPs. Email errors should be logged for visibility. These measures align with application security verification themes such as access control, authentication, logging, and secure configuration (OWASP Foundation, n.d.-a).

13. Database Requirement

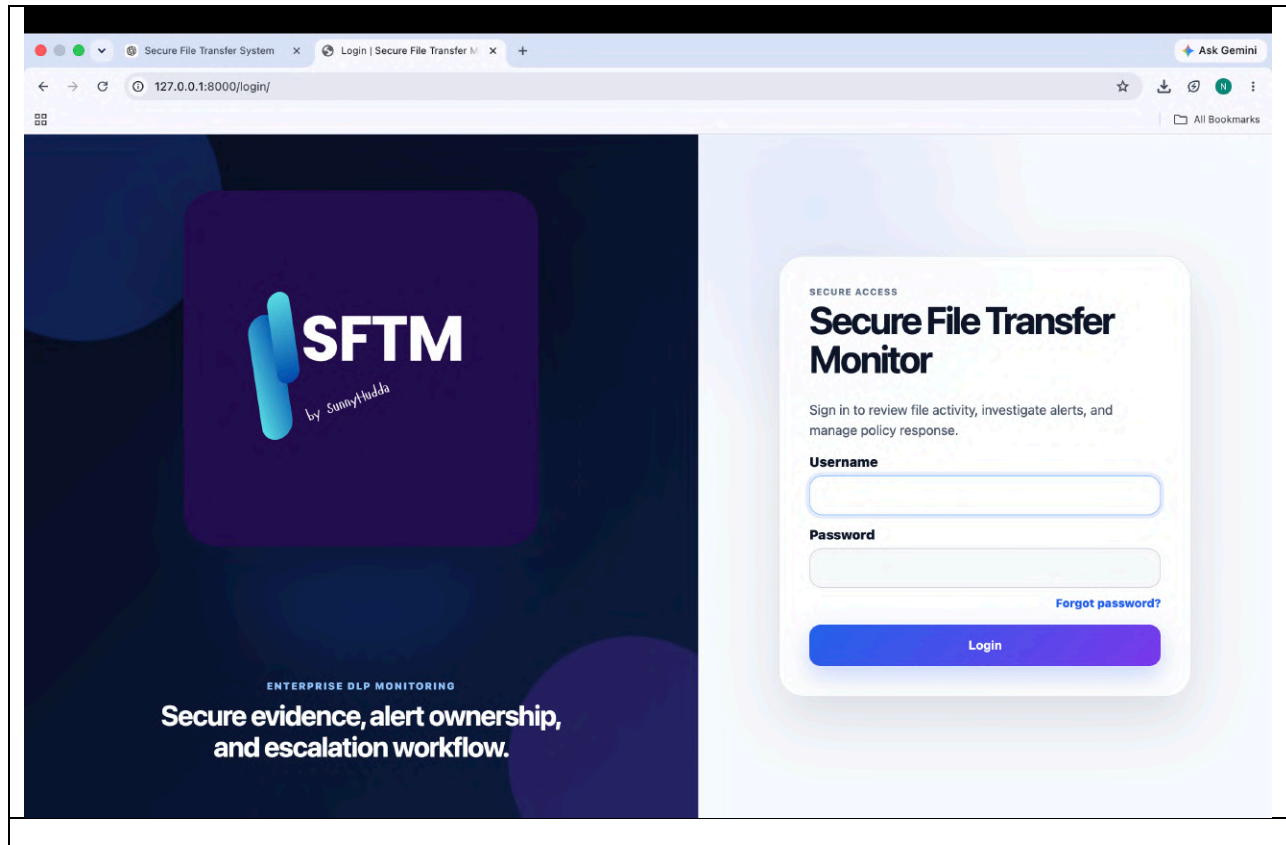
The project requires a database at runtime. In this implementation the runtime database is `db.sqlite3`, created by Django migrations. The database stores users, roles, profiles, policies, policy membership, file events, alert ownership, analyst notes, resolved status, email records, OTP records, integrity scan history, and report evidence. However, the database should not be submitted inside the project zip because it may contain test users, old alerts, email addresses, or private information.

SQLite is appropriate for this project because it is simple, local, and does not require a separate database server. Python documentation describes `sqlite3` as an interface to SQLite databases, and the SQLite project describes SQLite as a self-contained SQL database engine (Python Software Foundation, n.d.-c; SQLite Consortium, n.d.). If the system were converted to production use, PostgreSQL or another managed database would be more appropriate.

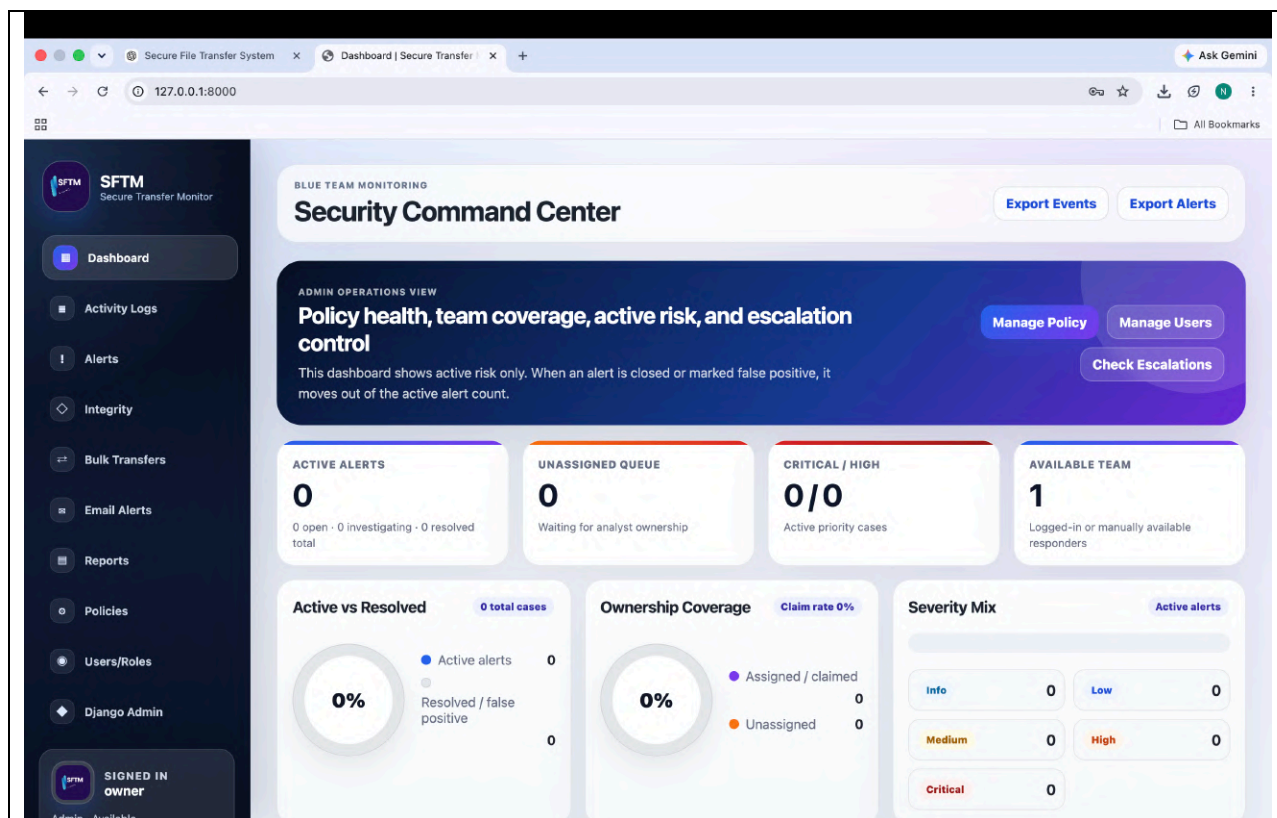
14. Screenshot Placeholders

The following placeholders are intentionally left for working screenshots. Screenshots should be added after the final video test so that they show the exact version of the application that was submitted.

Screenshot Placeholder 1. Login page with split-screen logo and username/password form



Screenshot Placeholder 2. Superuser dashboard showing command center view



Screenshot Placeholder 3. Users/Roles page showing created policy admin, analyst, and auditor

The screenshot displays the 'Users/Roles' page in the SFTM (Secure Transfer Monitor) application. The page is divided into two main sections: 'Create Project User' and 'Existing Users'.

Create Project User Section:

- Role-Based Access Control:** This section defines roles and their permissions. Roles include Super Admin, Policy Admin, Security Analyst, and Auditor.
- Form Fields:** The 'Create Project User' form includes fields for Username, Email, Role (dropdown), Availability status (dropdown), Password, and Password confirmation. A 'Create User' button is at the bottom.

Existing Users Section:

This section shows a table of existing users. The table has columns for Username, Email, Availability, Superuser status, Groups, Policy Memberships, and Password Reset status. There are 3 accounts listed.

USERNAME	EMAIL	AVAILABILITY	SUPERUSER	GROUPS	POLICY MEMBERSHIPS	PASSWORD RESET
admin1	nehalcanadaedu@gmail.com	Off Duty	No	Admin	Corporate Sensitive File Transfer Policy - Policy Admin	OTP Enabled
owner	sunnycanadaedu@gmail.com	Available	Yes	Unassigned	No policy scope	OTP Enabled
sunny	shanteeatasa@gmail.com	Available	No	Security Analyst	Corporate Sensitive File Transfer Policy - Security Analyst	OTP Enabled

Screenshot Placeholder 4. Policy creation page with monitored paths and escalation settings

The image displays two screenshots of the SFTM (Secure File Transfer Monitor) web application interface.

Top Screenshot: Monitoring Policies

The page title is "Monitoring Policies". It includes a sidebar with navigation links: Dashboard, Activity Logs, Alerts, Integrity, Bulk Transfers, Email Alerts, Reports, Policies (selected), Users/Roles, and Django Admin. The user is signed in as "owner".

The main content area shows a table of existing policies:

STATUS	POLICY	TEAM	ESCALATION	FOLDERS	HASH	UPDATED	ACTIONS
Active	Corporate Sensitive File Transfer Policy Monitors sensitive corporate files and alerts when restricted files are copied to USB, cl...	sunny - Security Analyst admin1 - Policy Admin	3 min SLA 2 group emails	1	SHA256	2026-06-17 17:07	Edit, Deactivate, Delete

Bottom Screenshot: Create Monitoring Policy

The page title is "Create Monitoring Policy". It includes the same sidebar and user information.

The main content area shows the "Policy Identity" section with the following fields:

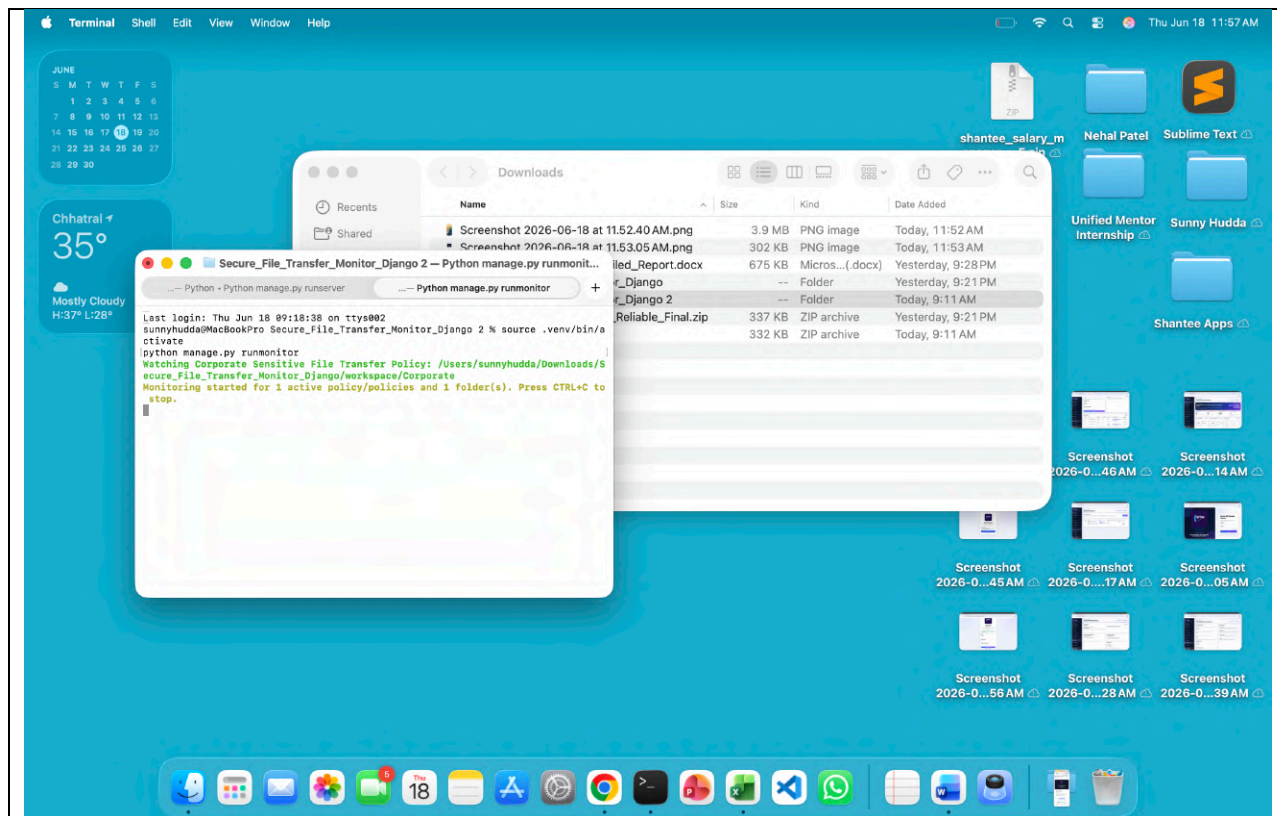
- Policy name:** My File Transfer Monitoring Policy
- Policy description:** Example: Monitors finance folders and blocks unauthorized USB/cloud movement.
- Make this the active monitoring policy:** ☒

The "1. File Transfer Logging Scope" section includes a text area for "Monitored directories" with the example path: /Users/yourname/SFTM/Workspace.

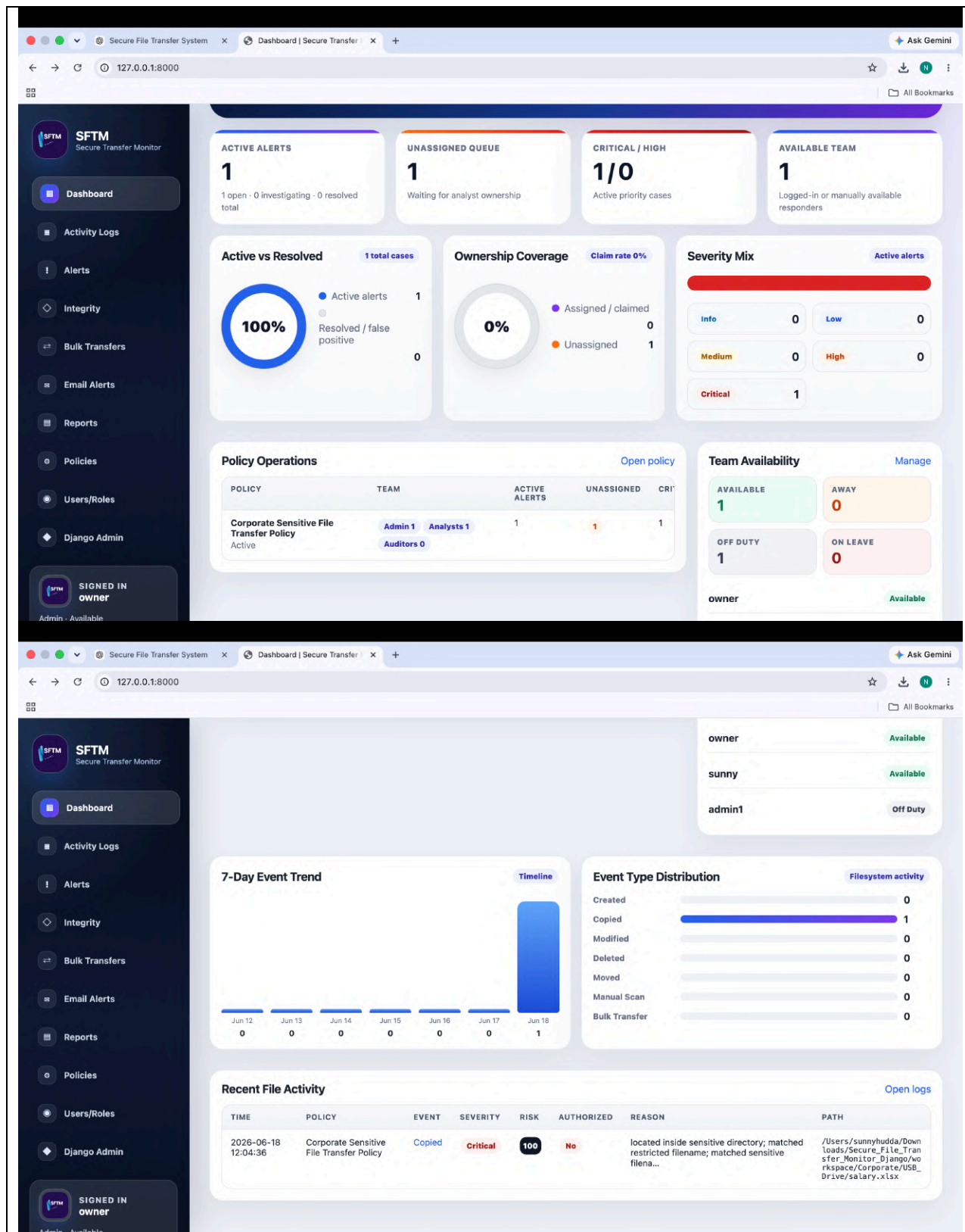
The "2. Sensitive Data Scope" section includes a text area for "Sensitive directories" with the example paths: /Users/yourname/SFTM/Workspace/Sensitive and /Users/yourname/SFTM/Workspace/Finance.

Below the sensitive directories section is a text area for "Restricted files" with the example filenames: salary.xlsx, client_data.csv, passwords.txt, and database_backup.sql.

Screenshot Placeholder 5. Terminal running runmonitor



Screenshot Placeholder 6. Generated alert on dashboard after copying sensitive file



Screenshot Placeholder 7. Simplified alert detail page

The image displays two screenshots of the SFTM (Secure File Transfer Monitor) web application interface.

Top Screenshot: Alert Queue

The browser address bar shows `127.0.0.1:8000/alerts/`. The page title is "Alert Queue". The left sidebar contains navigation links: Dashboard, Activity Logs, Alerts (selected), Integrity, Bulk Transfers, Email Alerts, Reports, Policies, Users/Roles, and Django Admin. The user is signed in as "Admin - Available".

The main content area shows the "Alert Queue" with a description: "Open and investigate alerts. The list shows the file, status, owner, and the plain-language reason first." There are buttons for "Export Events", "Export Alerts", and "Run Escalation Check".

Search filters include: "Search file, reason, path, or user", "All statuses", "All severities", "All ownership", "Filter", and "Reset".

Two alerts are listed:

- Alert 1:** Critical, Open. Path: `/Users/sunnyhudda/Downloads/Secure_File_Transfer_Monitor_Django/workspace/Corporate/...`. Reason: "matched restricted filename; matched sensitive filename keyword: salary; sensitive extension observed: .xlsx; Sensitive file movement matched USB/cloud/...". Risk: 100. Owner: Unassigned. Policy: Corporate Sensitive File Transfer Policy - 2026-06-18 12:04.
- Alert 2:** Critical, Open. Path: `/Users/sunnyhudda/Downloads/Secure_File_Transfer_Monitor_Django/workspace/Corporate/...`. Reason: "located inside sensitive directory; matched restricted filename; matched sensitive filename keyword: salary; sensitive extension observed: .xlsx; Sen...". Risk: 100. Owner: Unassigned. Policy: Corporate Sensitive File Transfer Policy - 2026-06-18 12:04.

Bottom Screenshot: Activity Logs

The browser address bar shows `127.0.0.1:8000/events/`. The page title is "Activity Logs". The left sidebar is the same as the top screenshot.

The main content area shows the "Activity Logs" with a description: "Search path, reason, category, user, process, source, destination, integrity, reason". There are buttons for "Filter" and "Reset".

Search filters include: "Search path, reason, category, user, process", "All severities", "All events", "Authorization", "Sensitivity", "Filter", and "Reset".

The activity log table has the following columns: TIME, EVENT, SEVERITY, RISK, CATEGORY, AUTHORIZED, USER, PROCESS, SOURCE, DESTINATION, INTEGRITY, and REASON.

Two events are listed:

- Event 1:** 2026-06-18 12:04:42, Modified, Critical, 100. Category: Keyword Classified Sensitive Data. Authorized: No. User: sunnyhudda. Process: Unknown. Source: `/Users/sunnyhudda/Downloads/Secure_File_Transfer_Monitor_Django/workspace/Corporate/USB_Drive/salary.xlsx`. Destination: `/Users/sunnyhudda/Downloads/Secure_File_Transfer_Monitor_Django/workspace/Corporate/USB_Drive/salary.xlsx`. Integrity: Hash unchanged. Reason: "matched restricted filename; matched sensitive filename keyword: salary; sensitive extension observed: .xlsx; Sensitive file movement matched USB/cloud/... blocked key modified".
- Event 2:** 2026-06-18 12:04:36, Copied, Critical, 100. Category: Keyword Classified Sensitive Data. Authorized: No. User: sunnyhudda. Process: Unknown. Source: `/Users/sunnyhudda/Downloads/Secure_File_Transfer_Monitor_Django/workspace/Corporate/USB_Drive/salary.xlsx`. Destination: `/Users/sunnyhudda/Downloads/Secure_File_Transfer_Monitor_Django/workspace/Corporate/USB_Drive/salary.xlsx`. Integrity: Hash unchanged. Reason: "located inside sensitive directory; matched restricted filename; matched sensitive filename keyword: salary; sensitive extension observed: .xlsx; Sensitive or restricted file appeared out approved destination; Sensitive file moved outside approved destination; Sensitive file movement matched USB/cloud/... blocked key".

The screenshot displays the SFTM (Secure File Transfer Monitor) web interface. The browser address bar shows the URL `127.0.0.1:8000/alerts/8/`. The interface is divided into a left sidebar and a main content area.

Left Sidebar:

- SFTM Secure Transfer Monitor
- Dashboard
- Activity Logs
- Alerts**
- Integrity
- Bulk Transfers
- Email Alerts
- Reports
- Policies
- Users/Roles
- Django Admin
- SIGNED IN owner (Admin - Available)

Main Content Area:

Alert Details:

- matched blocked keyword(s):** USB_Drive.
- DESTINATION:** /Users/sunnyhudda/Downloads/Secure_File_Transfer_Monitor_Django/workspace/Corporate/USB_Drive/salary.xlsx
- ORIGINAL SOURCE:** /Users/sunnyhudda/Downloads/Secure_File_Transfer_Monitor_Django/workspace/Corporate/Sensitive/salary.xlsx
- Description:** The monitor detected a newly created destination file and matched it to a source file with the same filename and hash in a sensitive folder.
- Buttons:** Claim Alert, Release

Why it was flagged:

- Matched rules: Restricted filename matched; Sensitive keyword matched; Sensitive file extension.
- Restricted filename matched**
- Sensitive keyword matched**
- Sensitive file extension**
- Sensitive directory rule**
- Blocked destination: USB_Drive**
- DESTINATION CHECK:** Destination matched blocked keyword(s): USB_Drive.

File evidence:

- File name:** salary.xlsx
- Detected location:** /Users/sunnyhudda/Downloads/Secure_File_Transfer_Monitor_Django/workspace/Corporate/USB_Drive/salary.xlsx
- Destination:** /Users/sunnyhudda/Downloads/Secure_File_Transfer_Monitor_Django/workspace/Corporate/USB_Drive/salary.xlsx
- Source:** /Users/sunnyhudda/Downloads/Secure_File_Transfer_Monitor_Django/workspace/Corporate/Sensitive/salary.xlsx
- System user:** sunnyhudda
- Process:** Unknown
- Buttons:** Full Event

Assign or reassign:

- Ownership:** Assign to analyst (dropdown menu)
- Save Assignment**

Investigation notes:

- Status:** Open
- Analyst notes:** (Text area)
- Recommended action:** Immediately review the file transfer, validate the destination, preserve evidence, contact the user/owner, and isolate the endpoint if exfiltration is suspected.
- Save Investigation**

Advanced evidence:

- Show technical evidence**

Email notifications:

- Open email log**

Screenshot Placeholder 8. Analyst claim and investigation notes

The image displays two screenshots of the SFTM (Secure File Transfer Monitor) Alert Queue interface, illustrating an analyst's claim and investigation notes.

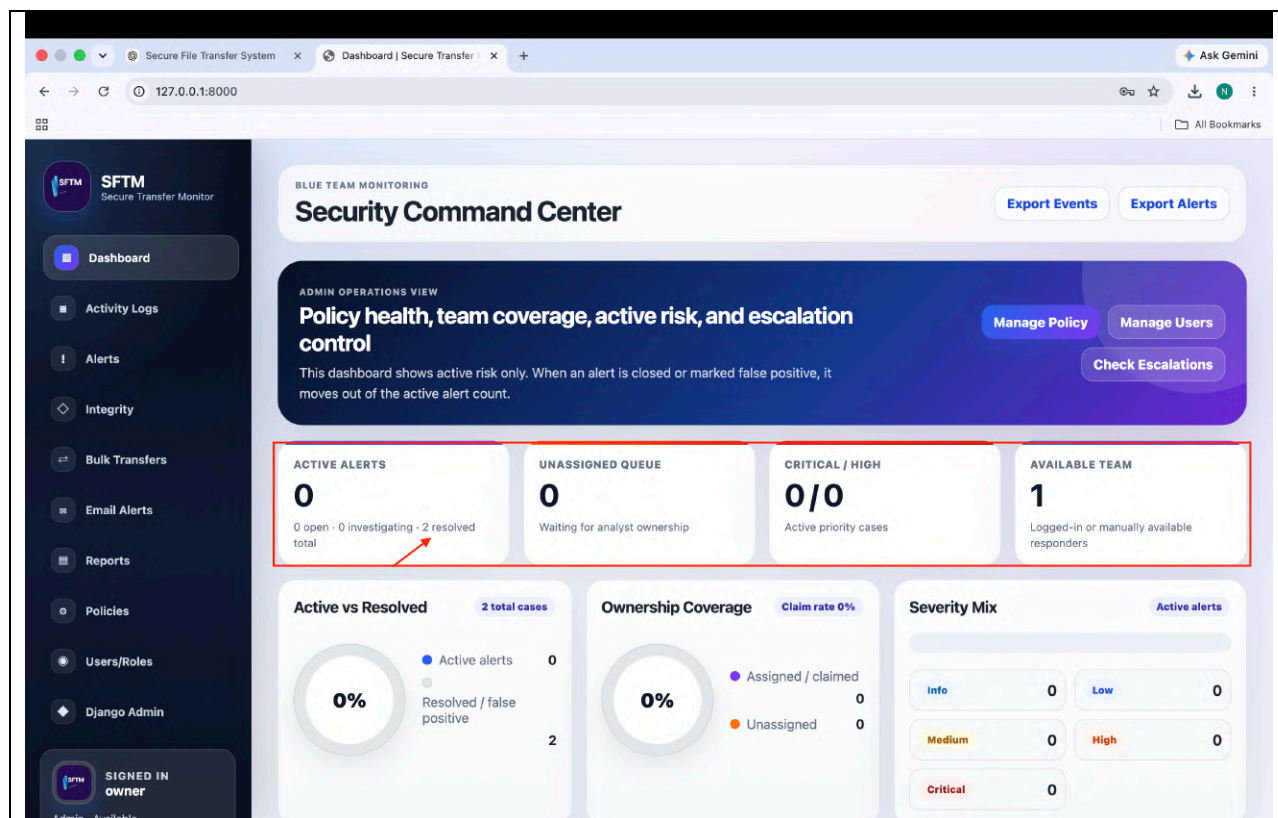
Top Screenshot: The interface shows the "Alert Queue" under "BLUE TEAM MONITORING". The alert status is "Open". The alert details include the file path `/Users/sunnyhudda/Downloads/Secure_File_Transfer_Monitor_Django/workspace/Corporate/...`, the reason "matched restricted filename; matched sensitive filename keyword: salary; sensitive extension observed: .xlsx; Sensitive file movement matched USB/clo...", and the policy "Corporate Sensitive File Transfer Policy - 2026-06-18 12:04". The risk level is "RISK 100" with the owner "Unassigned".

Bottom Screenshot: The same alert is shown, but the status has changed to "Closed". The alert details and risk level remain the same. The analyst's name "sunny" is visible in the bottom left corner, indicating they are signed in.

The screenshot displays the 'Analyst Investigation Workbench' interface within the 'Secure File Transfer System'. The browser's address bar shows the URL '127.0.0.1:8000'. The dashboard is organized into several sections:

- Left Sidebar:** Contains navigation links for 'Dashboard', 'Activity Logs', 'Alerts', 'Integrity', 'Bulk Transfers', 'Email Alerts', and 'Reports'. At the bottom, it shows the user is 'SIGNED IN sunny' and provides a 'Logout' button.
- Top Section:** Titled 'BLUE TEAM MONITORING Analyst Investigation Workbench', it includes 'Export Events' and 'Export Alerts' buttons. Below this, a purple banner states 'Owned investigations, claimable alerts, and evidence triage' with a 'YOUR STATUS Available' badge.
- Metrics Row:** Four summary cards are displayed:
 - MY ACTIVE CASES:** 1 (Assigned or claimed by you)
 - CLAIMABLE:** 0 (Unassigned alerts in your policy)
 - CRITICAL / HIGH:** 1/0 (Your policy scope)
 - CLOSED TODAY:** 1 (Completed investigations)
- Investigation Queue:** A section titled 'My Investigation Queue' with a 'View mine' link. It contains a critical alert: 'located inside sensitive directory; matched restricted filename; matched sensitive filena...' under the status 'Critical Investigating'. Below the alert, it specifies 'Corporate Sensitive File Transfer Policy - Risk 100 - 2026-06-18 12:04'.
- Claimable Alerts:** A section titled 'Claimable Alerts' with an 'Open queue' link, stating 'No unassigned alerts are waiting in your policy scope.'
- Bottom Section:** Includes a 'Case Status' chart showing '2 total' and 'Active 1', a 'Severity Bars' chart with 'Info', 'Low', and 'Medium' levels all at 0, and an 'Assigned Policy' section for 'Security Analyst' under the 'Corporate Sensitive File Transfer Policy'.

Screenshot Placeholder 9. Resolved alert with active count returned to zero



Screenshot Placeholder 10. Forgot password OTP email or Email Alerts page

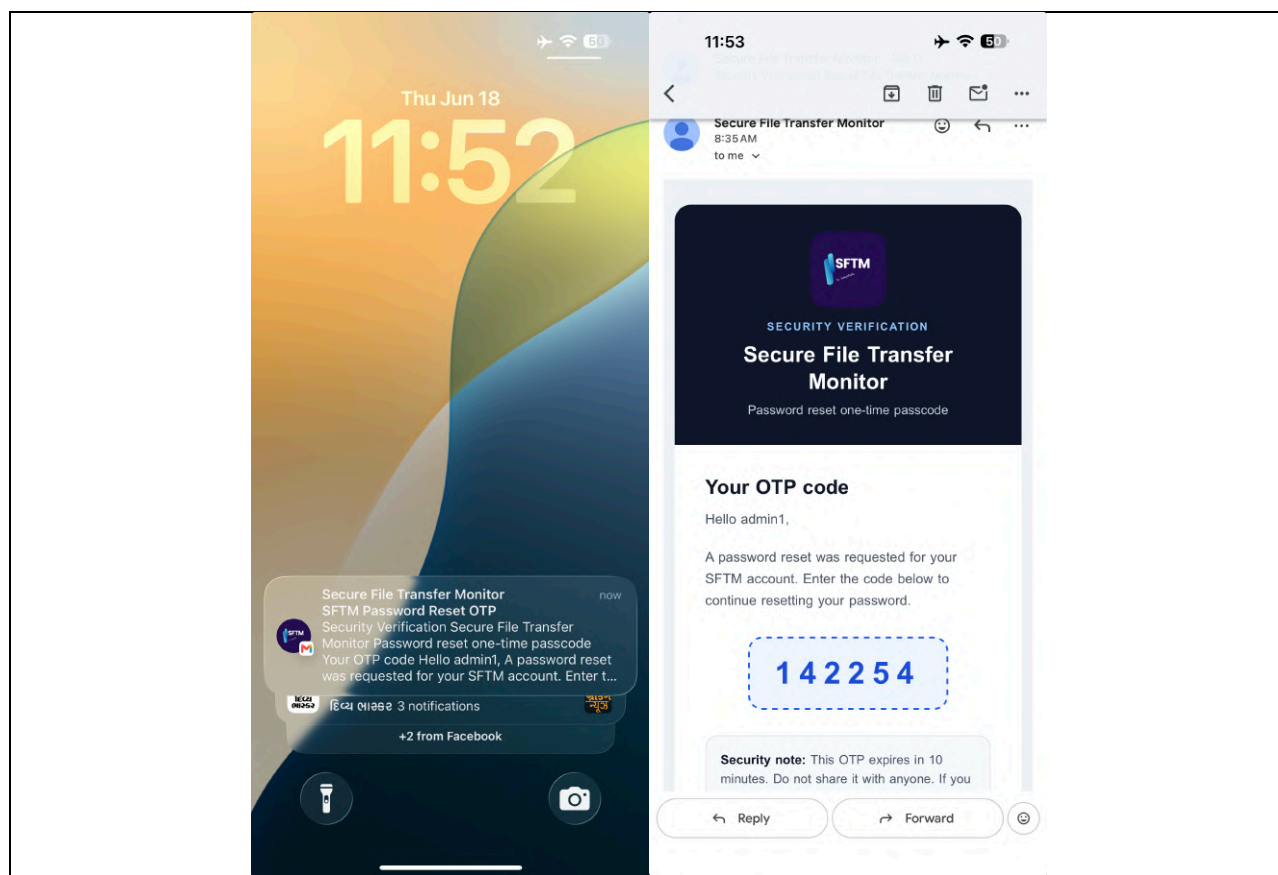
The image displays two sequential screenshots of a web application interface for password reset, titled "Secure File Transfer System".

Top Screenshot: Reset Password

- URL:** 127.0.0.1:8000/password-reset/
- Header:** Secure File Transfer System | Password Reset OTP | Secure
- Logo:** SFTM
- Title:** Reset Password
- Text:** Enter your account email. The system will email a one-time password reset code.
- Form:** Account email (nehalcanadaedu@gmail.com)
- Buttons:** Send OTP, Back to login

Bottom Screenshot: Enter OTP

- URL:** 127.0.0.1:8000/password-reset/verify/?email=nehalcanadaedu@gmail.com
- Header:** Secure File Transfer System | Verify OTP | Secure File Transfer
- Logo:** SFTM
- Title:** Enter OTP
- Text:** Enter the 6-digit code sent to nehalcanadaedu@gmail.com and choose a new password.
- Message:** If the email matches an active account, an OTP has been sent.
- Form:** Email, OTP code, New password, Confirm new password
- Buttons:** Reset Password, Request a new OTP



Screenshot Placeholder 11. Unassigned alert escalation email evidence

The screenshot displays two views of a security alert from the Secure File Transfer Monitor (SFTM) system.

Desktop View (Top):

- Browser:** The address bar shows the URL `127.0.0.1:8000/alerts/9/`.
- SFTM Dashboard:** The left sidebar shows the SFTM logo and navigation menu: Dashboard, Activity Logs, Alerts (selected), Integrity, Bulk Transfers, Email Alerts, Reports, Policies, Users/Roles, and Django Admin. The user is signed in as 'Admin - Available'.
- Email notifications:** The main content area shows an 'Escalation Notification' for a critical alert. The subject is '[ACTION REQUIRED] Critical - salary.xlsx - To sunnycanadaedu@gmail.com - Sent - 2026-06-18 12:08'. The body details the unclaimed alert escalation, including the file path `workspace/Corporate/USB_Drive/salary.xlsx`, the policy 'Corporate Sensitive File Transfer Policy', and the risk score '100/100 (Critical Risk)'. It also states the alert is 'Unassigned' and provides a required action: 'Assign it to an admin or analyst and review the file movement now.' A link to 'Open alert: http://127.0.0.1:8000/alerts/9/' is provided.

Mobile View (Bottom):

- App Interface:** The mobile app shows a 'NEW FILE TRANSFER ALERT' with the title 'Critical alert: salary.xlsx'. The alert is identified as 'Alert #8 - Corporate Sensitive File Transfer Policy' and was sent on '2026-06-18 12:04:36'.
- WHAT HAPPENED:** The alert states that 'salary.xlsx was copied from workspace/Corporate/Sensitive/salary.xlsx to workspace/Corporate/USB_Drive/salary.xlsx. Destination matched blocked keyword(s): USB_Drive.'
- SEVERITY:** The severity is 'Critical' with a risk score of '100/100 - Critical Risk'.
- OWNER / STATUS:** The owner is 'Unassigned' and the status is 'Open'.
- Key evidence:** A table shows the file 'salary.xlsx' and its destination path: `/Users/sunnyhudda/Downloads/Secure_File_Transfer_Monitor_Django/workspace/Corporate/USB_Drive/salary.xlsx`.
- REQUIRED ACTION:** A green box highlights the required action: 'Confirm whether this file movement was approved. If not approved, remove the file from the risky location and document the incident.'
- Matched policy rule(s):** A list of rules is shown: 'Restricted filename matched', 'Sensitive keyword matched', 'Sensitive file extension', 'Sensitive directory rule', and 'Blocked destination: USB_Drive'.
- Buttons:** At the bottom, there are buttons for 'Open alert in dashboard', 'Reply all', and 'Forward'.

15. Future Developments

Future development should focus on scalability, stronger detection, production security, and richer integrations. The current system is suitable for learning and demonstration, but real organizations would need centralized logging, endpoint integration, storage encryption, retention policies, and stronger identity controls.

Table 7

Recommended future improvements

Area	Future improvement	Reason
Detection	Add content inspection and file-type validation.	Filename and extension checks are useful but can be bypassed by renaming files.
Endpoint integration	Collect real process and user telemetry from endpoint agents.	Improves attribution and investigation quality.
Storage	Move from SQLite to PostgreSQL for multi-user deployment.	Improves concurrency, backup, and production reliability.
Authentication	Add MFA and stronger password policies.	Reduces risk of account compromise.
Notifications	Integrate Microsoft Teams, Slack, or Jira.	Improves SOC workflow and ticket tracking.
Reporting	Add executive PDF reports and CSV filtering.	Supports audit and management review.
Security	Add encryption for sensitive configuration and secrets management.	Protects SMTP credentials and sensitive operational settings.
Monitoring	Add live WebSocket updates for alerts.	Reduces need to refresh dashboard manually.

16. Conclusion

The Secure File Transfer Monitoring System demonstrates how policy-based file monitoring, alert ownership, email escalation, OTP password reset, and audit evidence can be combined into a single cybersecurity lab project. The project is valuable because it does not only generate alerts; it also supports the response workflow after an alert appears. A security analyst can claim a case, document action, resolve the alert, and preserve evidence for later review. The design is intentionally clean and explainable, making it suitable for a portfolio demonstration, classroom project, or entry-level cybersecurity interview discussion.

References

- Cichonski, P., Millar, T., Grance, T., & Scarfone, K. (2012). Computer security incident handling guide (NIST Special Publication 800-61 Revision 2). National Institute of Standards and Technology. <https://csrc.nist.gov/pubs/sp/800/61/r2/final>
- Django Software Foundation. (n.d.-a). Using the Django authentication system. Django documentation. Retrieved June 17, 2026, from <https://docs.djangoproject.com/en/6.0/topics/auth/default/>
- Django Software Foundation. (n.d.-b). Sending email. Django documentation. Retrieved June 17, 2026, from <https://docs.djangoproject.com/en/6.0/topics/email/>
- Django Software Foundation. (n.d.-c). Password management in Django. Django documentation. Retrieved June 17, 2026, from <https://docs.djangoproject.com/en/6.0/topics/auth/passwords/>
- Open Worldwide Application Security Project. (n.d.-a). OWASP Application Security Verification Standard (ASVS). Retrieved June 17, 2026, from <https://owasp.org/www-project-application-security-verification-standard/>
- Open Worldwide Application Security Project. (n.d.-b). OWASP Top Ten Web Application Security Risks. Retrieved June 17, 2026, from <https://owasp.org/www-project-top-ten/>
- Python Software Foundation. (n.d.-a). pathlib - Object-oriented filesystem paths. Python documentation. Retrieved June 17, 2026, from <https://docs.python.org/3/library/pathlib.html>
- Python Software Foundation. (n.d.-b). hashlib - Secure hashes and message digests. Python documentation. Retrieved June 17, 2026, from <https://docs.python.org/3/library/hashlib.html>

Python Software Foundation. (n.d.-c). sqlite3 - DB-API 2.0 interface for SQLite databases.

Python documentation. Retrieved June 17, 2026, from

<https://docs.python.org/3/library/sqlite3.html>

SQLite Consortium. (n.d.). SQLite documentation. Retrieved June 17, 2026, from

<https://sqlite.org/docs.html>